

[Dashboard](#) / [My courses](#) / [ITB_IF2010_2_2526](#) / [Tutorial 6](#) / [Exception, Assertion, Multithreading, Reflection](#)

Started on	Saturday, 16 May 2026, 11:08 AM
State	Finished
Completed on	Monday, 18 May 2026, 11:59 PM
Time taken	2 days 12 hours
Grade	390.00 out of 400.00 (98%)

Question 1

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: BankSystem.zip

Kamu diminta membangun sistem manajemen rekening bank sederhana menggunakan Java. Sistem ini harus menangani berbagai kondisi error dengan **custom exceptions**, baik *checked exception* (turunan **Exception**) maupun *unchecked exception* (turunan **RuntimeException**).

File **Main.java** (driver) dan **AccountNotFoundException.java** sudah disediakan dan **tidak boleh diubah**. Tugas kamu adalah mengimplementasikan file-file berikut:

- **InsufficientFundsException.java**
- **InvalidAmountException.java**
- **AccountFrozenException.java**
- **BankAccount.java**
- **Bank.java**

Kumpulkan [BankSystem.zip](#) yang berisi semua file **.java** yang kamu buat, termasuk **Main.java** dan **AccountNotFoundException.java**.

Spesifikasi**1. AccountNotFoundException (sudah disediakan sebagai referensi)**

Checked exception yang dilempar ketika akun dengan ID tertentu tidak ditemukan.

Pesan: "Akun tidak ditemukan: <accountId>"

2. InsufficientFundsException (checked exception)

Extends **Exception**. Dilempar ketika saldo rekening tidak mencukupi untuk penarikan.

Constructor	Pesan Exception
InsufficientFundsException (String accountId, long balance, long amount)	"Saldo tidak cukup di akun <accountId> (saldo: <balance>, diperlukan: <amount>)"

3. InvalidAmountException (unchecked exception)

Extends **RuntimeException**. Dilempar ketika jumlah yang diberikan tidak valid (≤ 0 untuk deposit/withdraw, < 0 untuk saldo awal).

Constructor	Pesan Exception
InvalidAmountException (long amount)	"Jumlah tidak valid: <amount>"

4. AccountFrozenException (checked exception)

Extends **Exception**. Dilempar ketika operasi dilakukan pada akun yang sedang dibekukan.

Constructor	Pesan Exception
AccountFrozenException (String accountId)	"Akun dibekukan: <accountId>"

5. Kelas BankAccount

Merepresentasikan sebuah rekening bank milik seorang nasabah.

Method / Constructor	Keterangan
BankAccount (String id, long initialBalance)	Membuat rekening baru. Lempar InvalidAmountException jika initialBalance < 0 .
String getId()	Mengembalikan ID rekening.
long getBalance()	Mengembalikan saldo rekening saat ini.
boolean isFrozen()	Mengembalikan true jika rekening sedang dibekukan.
void deposit(long amount)	Menambah saldo sebesar amount . Urutan pengecekan: amount $\leq 0 \rightarrow$ InvalidAmountException ; rekening beku $\rightarrow$ AccountFrozenException .

Method / Constructor	Keterangan
<code>void withdraw(long amount)</code>	Mengurangi saldo sebesar <code>amount</code> . Urutan pengecekan: <code>amount ≤ 0</code> → <code>InvalidAmountException</code> ; rekening beku → <code>AccountFrozenException</code> ; saldo kurang → <code>InsufficientFundsException</code> .
<code>void freeze()</code>	Membekukan rekening.
<code>void unfreeze()</code>	Mengaktifkan kembali rekening yang dibekukan.

6. Kelas Bank

Mengelola kumpulan rekening menggunakan `LinkedHashMap<String, BankAccount>`.

Method	Keterangan
<code>void addAccount(String id, long initialBalance)</code>	Membuat dan menyimpan rekening baru. <code>InvalidAmountException</code> dari konstruktor <code>BankAccount</code> ter-propagate otomatis.
<code>BankAccount getAccount(String id)</code>	Mengembalikan rekening dengan ID yang diberikan. Lempar <code>AccountNotFoundException</code> jika tidak ditemukan.
<code>void deposit(String id, long amount)</code>	Memanggil <code>deposit()</code> pada rekening yang sesuai. Propagate semua exception.
<code>void withdraw(String id, long amount)</code>	Memanggil <code>withdraw()</code> pada rekening yang sesuai. Propagate semua exception.
<code>void freeze(String id)</code>	Membekukan rekening. Lempar <code>AccountNotFoundException</code> jika tidak ditemukan.
<code>void unfreeze(String id)</code>	Mengaktifkan rekening. Lempar <code>AccountNotFoundException</code> jika tidak ditemukan.
<code>void transfer(String fromId, String toId, long amount)</code>	Mentransfer <code>amount</code> dari rekening <code>fromId</code> ke <code>toId</code> . Panggil <code>withdraw()</code> pada rekening asal lalu <code>deposit()</code> pada rekening tujuan. Propagate semua exception.

Format Masukan

Input terdiri atas sejumlah baris, masing-masing berisi satu perintah:

- `ADD <id> <saldo_awal>`: membuat rekening baru
- `DEPOSIT <id> <jumlah>`: setor uang ke rekening
- `WITHDRAW <id> <jumlah>`: tarik uang dari rekening
- `FREEZE <id>`: bekukan rekening
- `UNFREEZE <id>`: aktifkan rekening
- `TRANSFER <id_asal> <id_tujuan> <jumlah>`: transfer antar rekening
- `STATUS <id>`: tampilkan info rekening

Format Keluaran

Perintah	Kondisi	Keluaran
ADD	berhasil	Akun <id> dibuat dengan saldo <saldo>.
DEPOSIT	berhasil	Deposit <jumlah> ke akun <id> berhasil.
WITHDRAW	berhasil	Tarik <jumlah> dari akun <id> berhasil.
FREEZE	berhasil	Akun <id> dibekukan.
UNFREEZE	berhasil	Akun <id> diaktifkan.
TRANSFER	berhasil	Transfer <jumlah> dari <id_asal> ke <id_tujuan> berhasil.
STATUS	-	Akun <id>: Saldo=<saldo>, Status=<Aktif/Beku>.
Semua perintah	terjadi exception	Error: <pesan exception>

Contoh Masukan dan Keluaran

Masukan 1:

```
ADD A001 1000000
ADD A002 500000
ADD A003 0
```

```
DEPOSIT A001 200000
DEPOSIT A002 100000
STATUS A001
STATUS A002
STATUS A003
```

Keluaran 1:

Akun A001 dibuat dengan saldo 1000000.
Akun A002 dibuat dengan saldo 500000.
Akun A003 dibuat dengan saldo 0.
Deposit 200000 ke akun A001 berhasil.
Deposit 100000 ke akun A002 berhasil.
Akun A001: Saldo=1200000, Status=Aktif.
Akun A002: Saldo=600000, Status=Aktif.
Akun A003: Saldo=0, Status=Aktif.

Masukan 2:

```
ADD D001 800000
ADD D002 200000
FREEZE D001
DEPOSIT D001 100000
WITHDRAW D001 50000
TRANSFER D001 D002 100000
STATUS D001
UNFREEZE D001
DEPOSIT D001 100000
STATUS D001
```

Keluaran 2:

Akun D001 dibuat dengan saldo 800000.
Akun D002 dibuat dengan saldo 200000.
Akun D001 dibekukan.
Error: Akun dibekukan: D001
Error: Akun dibekukan: D001
Error: Akun dibekukan: D001
Akun D001: Saldo=800000, Status=Beku.
Akun D001 diaktifkan.
Deposit 100000 ke akun D001 berhasil.
Akun D001: Saldo=900000, Status=Aktif.

Masukan 3:

```
ADD E001 500000
DEPOSIT E001 -50000
DEPOSIT E001 0
WITHDRAW E001 -10000
ADD E002 -100
STATUS E001
```

Keluaran 3:

Akun E001 dibuat dengan saldo 500000.
Error: Jumlah tidak valid: -50000
Error: Jumlah tidak valid: 0
Error: Jumlah tidak valid: -10000
Error: Jumlah tidak valid: -100
Akun E001: Saldo=500000, Status=Aktif.

Java 8

 [ans1.zip](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	16	Accepted	0.07 sec, 28.92 MB
2	16	Accepted	0.07 sec, 28.95 MB
3	16	Accepted	0.06 sec, 30.91 MB
4	16	Accepted	0.06 sec, 26.35 MB
5	16	Accepted	0.06 sec, 28.12 MB
6	20	Accepted	0.06 sec, 30.97 MB

Question **2**

Partially correct

Mark 90.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Bantulah untuk membuat kelas DigitalClock ini lebih safe dengan memerhatikan pre & post condition pada suatu method menggunakan konsep Exception & Assertion. [Berikut](#) adalah file yang dibutuhkan. Kumpulkan file DigitalClock.java.

Java 8

 [DigitalClock.java](#)

Score: 90

Blackbox

Score: 90

Verdict: Wrong answer

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.06 sec, 28.97 MB
2	10	Accepted	0.11 sec, 28.02 MB
3	10	Accepted	0.07 sec, 28.59 MB
4	10	Accepted	0.07 sec, 29.00 MB
5	10	Accepted	0.07 sec, 28.42 MB
6	10	Accepted	0.06 sec, 28.03 MB
7	10	Accepted	0.06 sec, 28.12 MB
8	10	Accepted	0.07 sec, 28.29 MB
9	10	Accepted	0.06 sec, 27.89 MB
10	0	Wrong answer	0.06 sec, 28.60 MB

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Kebin sedang mengerjakan proyek MBG (Massive Binary Grid) untuk Pak Owo, sebuah sistem komputasi yang digunakan untuk memproses data matriks berukuran besar. Salah satu operasi utama dalam proyek tersebut adalah melakukan perkalian dua matriks.

Namun, implementasi perkalian matriks yang Kebin buat saat ini masih menggunakan pendekatan single-threaded sehingga proses komputasi menjadi sangat lambat ketika ukuran matriks semakin besar.

Untuk mempercepat proses perhitungan, Kebin membutuhkan bantuanmu untuk mengimplementasikan perkalian matriks menggunakan **10 thread yang berjalan secara paralel**.

Implementasikan kelas dalam file [berikut](#)

Untuk membantu melakukan pengujian, silahkan gunakan [Main.java](#)

Kumpulkan kembali file `MatrixMultiplication.java`

Java 8

 [MatrixMultiplication.java](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.07 sec, 28.80 MB
2	20	Accepted	0.07 sec, 29.18 MB
3	20	Accepted	0.07 sec, 29.73 MB
4	20	Accepted	0.06 sec, 29.05 MB
5	20	Accepted	0.07 sec, 28.96 MB

Question **4**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Bayangkan kalian sedang membangun aplikasi yang perlu menyimpan data ke database. Tanpa bantuan apapun, kalian harus menulis SQL secara manual untuk setiap tabel buat INSERT, buat DELETE, buat SELECT, dan seterusnya.

Solusi dari masalah ini adalah **ORM (Object-Relational Mapper)**. ORM adalah sebuah lapisan yang "menerjemahkan" objek ke operasi database secara otomatis. Kamu cukup mendefinisikan kelas seperti biasa, dan ORM yang akan menghasilkan SQL-nya.

Bagaimana ORM tahu nama tabelnya apa? Kolom mana yang jadi primary key? Jawabannya adalah **Annotation**. Pengguna cukup "mendekorasi" kelas dan field-nya dengan annotation khusus.

ORM akan membaca dekorasi-dekorasi itu secara *runtime* menggunakan **Reflection**, kemampuan Java untuk menginspeksi dan memanipulasi kelas, field, dan method sebuah objek pada saat program berjalan, tanpa perlu mengetahui struktur kelas tersebut saat kompilasi.

Buatlah sebuah mini ORM yang menggunakan Java Reflection untuk membaca annotation dan menjalankan lifecycle hooks secara otomatis.

File-file berikut sudah tersedia dan **tidak boleh diubah**.

- [TableName.java](#) — annotation yang diletakkan di atas deklarasi kelas. Menyimpan satu atribut: nama tabel (String value).
- [ColumnName.java](#) — annotation yang diletakkan di atas deklarasi field. Menyimpan dua atribut: nama kolom (String value) dan apakah kolom ini adalah primary key (boolean primaryKey, default false).
- [Hook.java](#) — annotation yang diletakkan di atas deklarasi method. Menentukan kapan method tersebut dipanggil, melalui atribut bertipe enum Hook.When yang punya tiga nilai: PRE_INSERT, POST_LOAD, dan PRE_DELETE.
- [Mahasiswa.java](#) — contoh kelas model. Punya tiga field beranotasi @ColumnName dan satu field biasa tanpa anotasi (catatan). Juga punya tiga lifecycle hook methods (POST_LOAD, PRE_INSERT, PRE_DELETE) yang implementasinya sudah ada di dalamnya — kamu cukup memanggilnya via Reflection.
- [Produk.java](#) — contoh kelas model lain. Punya tiga field beranotasi @ColumnName. Juga punya tiga lifecycle hook methods yang implementasinya sudah ada.
- [Main.java](#) — driver program yang membaca perintah dari stdin dan memanggil method-method di kelas ORM. Tidak perlu diubah.

Lengkapi dan kumpulkan file [ORM.java](#) yang sudah memiliki keterangannya masing-masing. Gunakan file main untuk mengetesnya.

[Contoh Input](#) & [Contoh Output](#)

Java 8

 [ORM.java](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	12	Accepted	0.08 sec, 28.21 MB
2	12	Accepted	0.19 sec, 34.64 MB

No	Score	Verdict	Description
3	12	Accepted	0.17 sec, 35.90 MB
4	12	Accepted	0.17 sec, 33.95 MB
5	12	Accepted	0.18 sec, 34.32 MB
6	12	Accepted	0.18 sec, 34.03 MB
7	12	Accepted	0.18 sec, 33.95 MB
8	16	Accepted	0.18 sec, 33.91 MB

[◀ Post Praktikum 5](#)[Slide Tutorial 6 ▶](#)